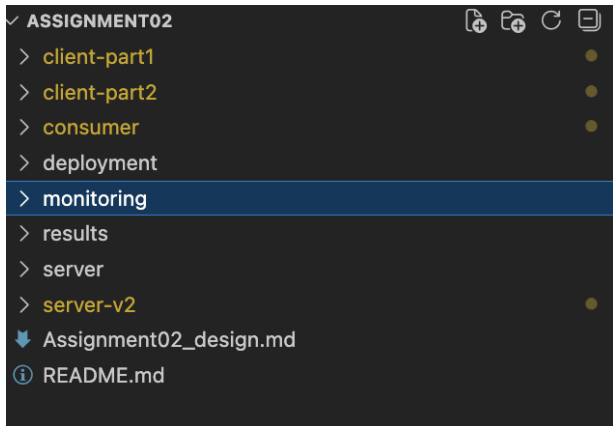


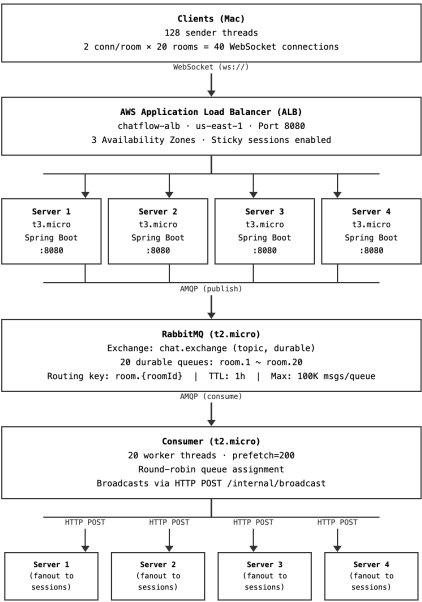
# 1. Git Repository URL

<https://github.com/fanzhangvivian/cs6650-Project/tree/main/Assignment02>

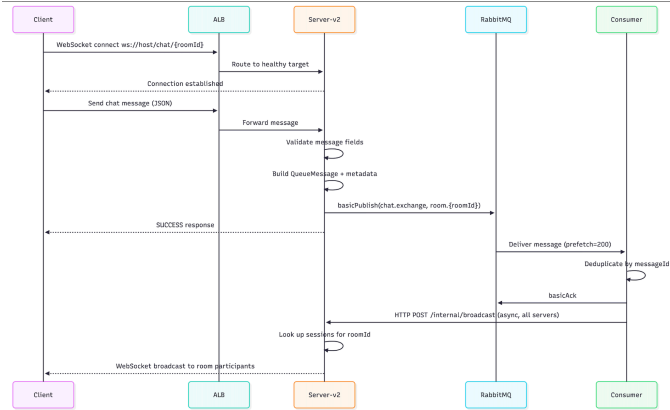


# 2. Architecture Document

## 1. System Architecture Diagram



## 2. Message Flow Sequence Diagram



## 3. Queue Topology Design

```
Exchange: chat.exchange (type=topic, durable=true)

Binding pattern:
room.1 —binding(room.1)→ queue: room.1
room.2 —binding(room.2)→ queue: room.2
...
room.20 —binding(room.20)→ queue: room.20

Queue configuration (per queue):
- Type: classic, durable
- x-message-ttl: 3,600,000 ms (1 hour)
- x-max-length: 100,000 messages
- Auto-delete: false
- Exclusive: false

Total: 20 queues × 100K max = 2M message capacity
```

## 4. Consumer Threading Model

```
Consumer Application (t2.micro)
├── Worker Thread 0  →  room.1,  room.3,  room.5, ... (odd rooms)
├── Worker Thread 1  →  room.2,  room.4,  room.6, ... (even rooms)
├── Worker Thread 2  →  room.1,  room.3,  ...
├── ...
└── Worker Thread 19 →  room.20, ...

Distribution: Round-robin assignment
workerIndex = (roomId - 1) % consumerThreadCount
e.g., 20 queues / 20 threads = 1 queue per thread (at thread count=20)

Per-thread configuration:
- Each thread owns exactly ONE Channel (thread-safe, no sharing)
- basicQos(prefetch=200): each thread buffers up to 200 unacked messages
- Manual acknowledgment (autoAck=false)
- ACK sent before HTTP broadcast (decouples queue from network latency)

Deduplication:
- ConcurrentHashMap tracks recent messageIds
- Cleared when size exceeds 10,000 to prevent unbounded memory growth
```

5. Load Balancing Configuration

| Parameter                | Value                           |
|--------------------------|---------------------------------|
| Load balancer type       | Application Load Balancer (ALB) |
| Scheme                   | Internet-facing                 |
| Listener protocol / port | HTTP:80                         |
| Target group             | chatflow-tg                     |
| Target port              | 8080                            |
| Availability Zones       | us-east-1b, us-east-1d          |
| Target group stickiness  | Off                             |
| Health check path        | /health                         |
| Health check protocol    | HTTP                            |
| Success codes            | 200                             |

6. Failure Handling Strategies

6.1 Server-side: Circuit Breaker (RabbitMQ publish)

```
States: CLOSED → OPEN → HALF_OPEN → CLOSED

- CLOSED: Normal operation, all publishes allowed
- OPEN:   RabbitMQ unavailable, publishes rejected immediately
         Returns ERROR to client (no queue wait)
- HALF_OPEN: After cooldown, allows trial publish
              Success → CLOSED, Failure → OPEN

Benefit: Prevents thread pile-up when RabbitMQ is down
```

6.2 Server-side: Channel Pool with auto-recovery

```
- ChannelPool pre-creates N channels on startup
- borrowChannel() / returnChannel() with thread-safe BlockingQueue
- returnChannel() checks isOpen() - broken channels are discarded
  and replaced with fresh channels automatically
- RabbitMQ connection has setAutomaticRecoveryEnabled(true)
  with 5-second retry interval for network failures
```

6.3 Consumer-side: basicNack on processing failure

```
- Happy path: basicAck after successful HTTP broadcast dispatch
- Error path: basicNack(requeue=false) - message dropped, not requeued
              (prevents poison message loops)
- Async HTTP: Broadcast failures are logged but do not block consumer
              thread or affect message acknowledgment
```

6.4 Client-side: Retry with exponential backoff

```
- Max retries: 5 attempts per message
- Initial backoff: 50ms, doubles each retry
- Connection timeout: 6,000ms
- Failed messages tracked and reported in final metrics
```

# 3. Test Results

## 3.1 Single Instance Tests

### 3.1.1 Single Instance client output

Phase 1: Warmup

Starting 32 threads to send 32000 messages...

MessageGenerator initialized with 50 predefined messages  
MessageGenerator started - generating 32000 messages...  
MessageGenerator completed: 32000 messages in 72ms

Warmup Phase Completed:  
Duration: 96878 ms  
Total messages sent: 32000  
Successful messages: 32000  
Failed messages: 0  
Throughput: 330.31 msg/sec  
Active connections: 40

Total connections created so far: 40

Phase 2: Main Load

Starting 128 threads to send 468000 messages...

MessageGenerator initialized with 50 predefined messages  
MessageGenerator started - generating 468000 messages...  
Generated: 50000 messages  
Generated: 100000 messages  
Generated: 150000 messages  
Generated: 200000 messages  
Generated: 250000 messages  
Generated: 300000 messages  
Generated: 350000 messages  
Generated: 400000 messages  
Generated: 450000 messages  
MessageGenerator completed: 468000 messages in 323910ms

Main Phase Completed:  
Total messages sent: 468000  
Duration: 414668 ms  
Throughput: 1128.61 msg/sec  
Active connections: 40

Total connections created: 40

Overall Test Results

Test Summary  
Successful messages: 500000  
Failed messages: 0  
Total runtime: 512578 ms  
Total connections: 40

Test Summary  
Successful messages: 500000  
Failed messages: 0  
Total runtime: 512578 ms  
Total connections: 40  
Reconnections: 0

Phase Breakdown:  
Warmup: 96878 ms (330.31 msg/sec)  
Main Load: 414668 ms (1128.61 msg/sec)

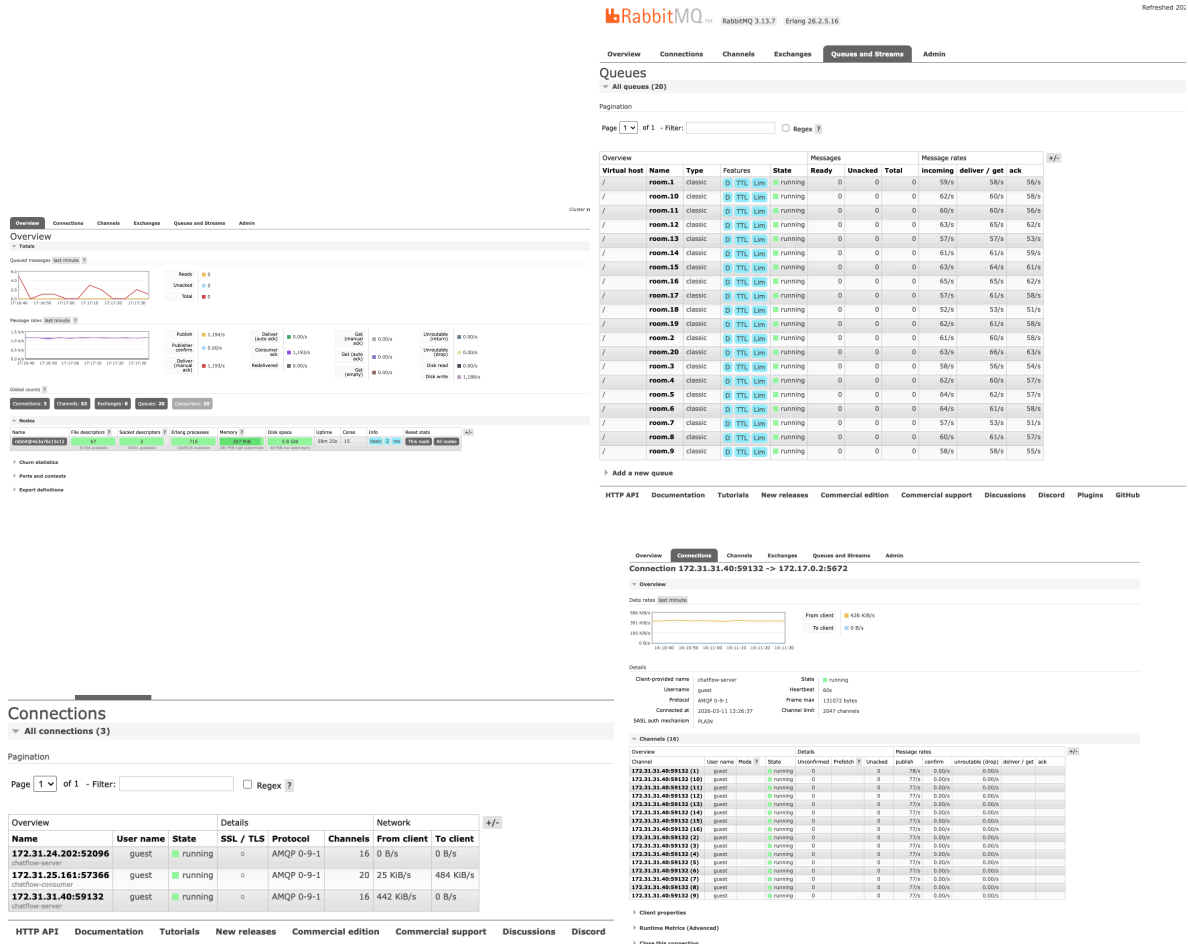
Detailed Performance Analysis

Performance Statistics  
Response Time Statistics (successful messages):  
Samples: 500000  
Mean response time: 111 ms  
Median response time: 110 ms  
95th percentile: 149 ms  
99th percentile: 177 ms  
Min response time: 68 ms  
Max response time: 852 ms

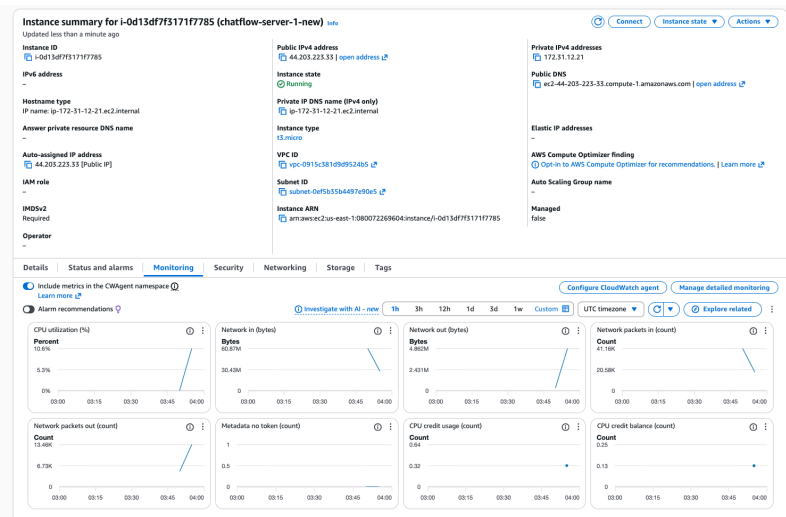
Throughput per Room  
Room 1: 25093 messages  
Room 2: 25021 messages  
Room 3: 25151 messages  
Room 4: 24869 messages  
Room 5: 25063 messages  
Room 6: 25116 messages  
Room 7: 24934 messages  
Room 8: 24783 messages  
Room 9: 24935 messages  
Room 10: 25019 messages  
Room 11: 24928 messages  
Room 12: 25054 messages  
Room 13: 24949 messages  
Room 14: 24854 messages  
Room 15: 24954 messages  
Room 16: 24991 messages  
Room 17: 25106 messages  
Room 18: 24803 messages  
Room 19: 25168 messages  
Room 20: 25217 messages

Message Type Distribution  
TEXT: 449854 messages (89.97%)  
JOIN: 25064 messages (5.01%)  
LEAVE: 25082 messages (5.02%)

### 3.1.2 Single Instance RabbitMQ Management Console showing



### 3.1.3 Single Instance EC2 System Metrics



**Key Takeaway:** The single-instance baseline test sent 500,000 messages using 40 WebSocket connections (2 per room × 20 rooms) and 128 sender threads. All 500,000 messages were delivered successfully with 0 failures. Main phase throughput was 1,128 msg/sec with a mean response time of 111 ms. RabbitMQ queue depth remained consistently near zero throughout the test, with publish rate at approximately 1,194/s and consumer ack rate matching at 1,193/s, indicating the consumer kept pace with the producer at all times

# 3.2 Load Balanced Tests

## 3.2.1 2 Instance 500k Client Output

Phase 1: Warmup

Starting 32 threads to send 32000 messages...

MessageGenerator initialized with 50 predefined messages  
MessageGenerator started - generating 32000 messages...  
MessageGenerator completed: 32000 messages in 75ms

Warmup Phase Completed:  
Duration: 96302 ms  
Total messages sent: 32000  
Successful messages: 32000  
Failed messages: 0  
Throughput: 332.29 msg/sec  
Active connections: 40

Total connections created so far: 40

Phase 2: Main Load

Starting 128 threads to send 468000 messages...

MessageGenerator initialized with 50 predefined messages  
MessageGenerator started - generating 468000 messages...  
Generated: 50000 messages  
Generated: 100000 messages  
Generated: 150000 messages  
Generated: 200000 messages  
Generated: 250000 messages  
Generated: 300000 messages  
Generated: 350000 messages  
Generated: 400000 messages  
Generated: 450000 messages  
MessageGenerator completed: 468000 messages in 315913ms

Main Phase Completed:  
Total messages sent: 468000  
Duration: 403798 ms  
Throughput: 1159.00 msg/sec  
Active connections: 40

Total connections created: 40

Overall Test Results

Test Summary  
Successful messages: 500000  
Failed messages: 0  
Total runtime: 501149 ms  
Total connections: 40  
Reconnections: 0

Phase Breakdown:  
Warmup: 96302 ms (332.29 msg/sec)  
Main Load: 403798 ms (1159.00 msg/sec)

Detailed Performance Analysis

Performance Statistics (successful messages):  
Response Time Statistics  
Samples: 500000  
Mean response time: 108 ms  
Median response time: 107 ms  
95th percentile: 145 ms  
99th percentile: 155 ms  
Min response time: 67 ms  
Max response time: 674 ms

Throughput per Room  
Room 1: 24708 messages  
Room 2: 24933 messages  
Room 3: 24757 messages  
Room 4: 24900 messages  
Room 5: 24936 messages  
Room 6: 24883 messages  
Room 7: 25251 messages  
Room 8: 25008 messages  
Room 9: 25018 messages  
Room 10: 25067 messages  
Room 11: 25076 messages  
Room 12: 25059 messages  
Room 13: 25106 messages  
Room 14: 24889 messages  
Room 15: 25100 messages  
Room 16: 24886 messages  
Room 17: 25077 messages  
Room 18: 25071 messages  
Room 19: 24996 messages  
Room 20: 25237 messages

Message Type Distribution  
TEXT: 450105 messages (90.02%)  
JSON: 24805 messages (4.9%)  
LEAVE: 25030 messages (5.01%)

## 3.2.1 2 Instance 500k RabbitMQ Management Console showing

RabbitMQ 3.13.7 Erlang 26.2.5.16

Overview Connections Channels Exchanges Queues and Streams Admin

Overview

Queued messages 386 (RabbitMQ)

Ready 0  
Unready 0  
Total 0

Message rates (RabbitMQ)

Global statistics

Connections 3 Channels 93 Exchanges 8 Queues 20 Connections 26

Nodes

Node statistics

Queues

Queues (20)

Overview

Virtual host Name Type Features State Messages Message rates

room.1 classic 0 TTL LRM in running 0 0 0 54/s 57/s 57/s

room.10 classic 0 TTL LRM in running 0 0 0 58/s 57/s 57/s

room.11 classic 0 TTL LRM in running 0 0 0 55/s 57/s 57/s

room.12 classic 0 TTL LRM in running 0 0 0 58/s 55/s 55/s

room.13 classic 0 TTL LRM in running 0 0 0 54/s 57/s 57/s

room.14 classic 0 TTL LRM in running 0 0 0 60/s 60/s 60/s

room.15 classic 0 TTL LRM in running 0 0 0 60/s 59/s 60/s

room.16 classic 0 TTL LRM in running 0 0 0 55/s 58/s 58/s

room.17 classic 0 TTL LRM in running 0 0 0 60/s 60/s 60/s

room.18 classic 0 TTL LRM in running 0 0 0 53/s 58/s 58/s

room.19 classic 0 TTL LRM in running 0 0 0 60/s 62/s 62/s

room.2 classic 0 TTL LRM in running 0 0 0 58/s 57/s 58/s

room.20 classic 0 TTL LRM in running 0 0 0 67/s 63/s 63/s

room.3 classic 0 TTL LRM in running 0 0 0 61/s 61/s 61/s

room.4 classic 0 TTL LRM in running 0 0 0 59/s 59/s 59/s

room.5 classic 0 TTL LRM in running 0 0 0 61/s 57/s 57/s

room.6 classic 0 TTL LRM in running 0 0 0 55/s 53/s 53/s

room.7 classic 0 TTL LRM in running 0 0 0 52/s 58/s 58/s

room.8 classic 0 TTL LRM in running 0 0 0 62/s 60/s 60/s

room.9 classic 0 TTL LRM in running 0 0 0 60/s 58/s 59/s

RabbitMQ 3.13.7 Erlang 26.2.5.16

Overview Connections Channels Exchanges Queues and Streams Admin

Connections

All connections (3)

Page 1 of 1 - Filter:  ☐ Regex

Overview

Details

Network

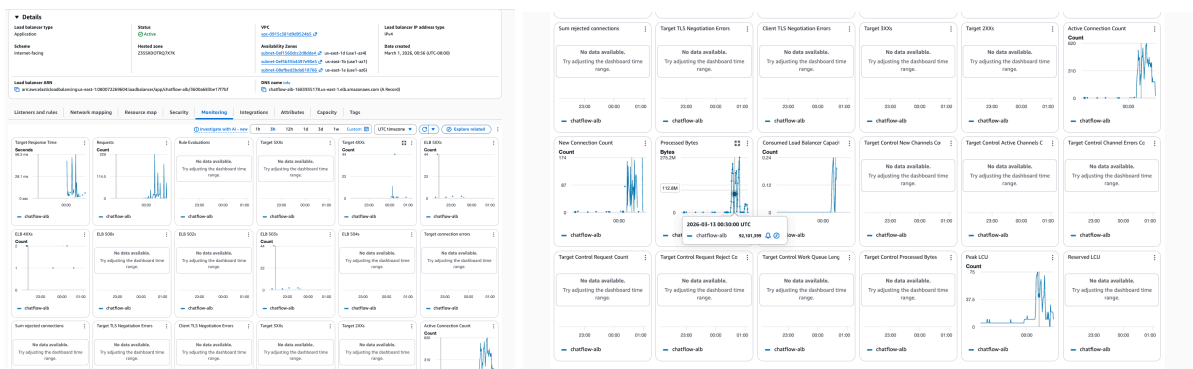
172.31.24.202:56968 guest running 0 AMQP 0-9-1 16 222 KiB/s 0 B/s

172.31.25.161:38064 guest running 0 AMQP 0-9-1 20 24 KiB/s 457 KiB/s

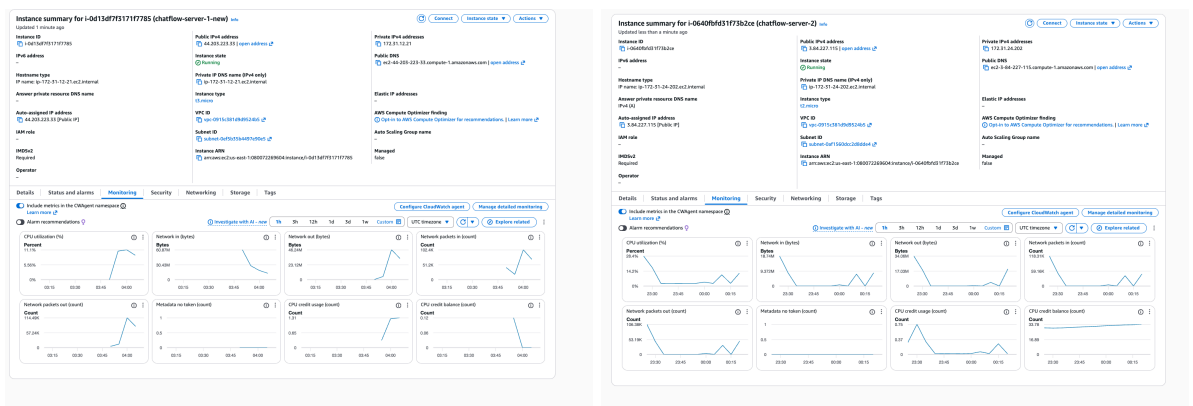
172.31.31.40:55626 guest running 0 AMQP 0-9-1 16 191 KiB/s 0 B/s

HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins GitHub

3.2.1 2 Instance 500k ALB metric



3.2.1 2 Instance 500k EC2 System Metric



3.2.2 4 Instance 500k Client Output

Phase 1: Warmup

Starting 32 threads to send 32000 messages...  
  
MessageGenerator initialized with 50 predefined messages  
MessageGenerator started - generating 32000 messages...  
MessageGenerator completed: 32000 messages in 75ms  
  
Warmup Phase Completed:  
Duration: 89730 ms  
Total messages sent: 32000  
Successful messages: 32000  
Failed messages: 0  
Throughput: 356.63 msg/sec  
Active connections: 40  
  
Total connections created so far: 40

Phase 2: Main Load

Starting 128 threads to send 468000 messages...  
  
MessageGenerator initialized with 50 predefined messages  
MessageGenerator started - generating 468000 messages...  
Generated: 50000 messages  
Generated: 100000 messages  
Generated: 150000 messages  
Generated: 200000 messages  
Generated: 250000 messages  
Generated: 300000 messages  
Generated: 350000 messages  
Generated: 400000 messages  
Generated: 450000 messages  
MessageGenerator completed: 468000 messages in 384800ms  
  
Main Phase Completed:  
Total messages sent: 468000  
Duration: 388970 ms  
Throughput: 1203.18 msg/sec  
Active connections: 40  
  
Total connections created: 40

Overall Test Results

Test Summary  
Successful messages: 468000  
Failed messages: 0  
Total runtime: 470728 ms  
Total connections: 40  
Reconnections: 0  
  
Phase Breakdown:  
Warmup: 89730 ms (356.63 msg/sec)  
Main Load: 388970 ms (1203.18 msg/sec)  
  
Detailed Performance Analysis

Performance Statistics  
Response Time Statistics (successful messages):  
Samples: 500000  
Mean response time: 104 ms  
Median response time: 98 ms  
95th percentile: 148 ms  
99th percentile: 157 ms  
Min response time: 68 ms  
Max response time: 1298 ms  
  
Throughput per Room  
Room 1: 25920 messages  
Room 2: 24851 messages  
Room 3: 24855 messages  
Room 4: 25157 messages  
Room 5: 25394 messages  
Room 6: 25835 messages  
Room 7: 25952 messages  
Room 8: 24862 messages  
Room 9: 24924 messages  
Room 10: 24871 messages  
Room 11: 24951 messages  
Room 12: 24884 messages  
Room 13: 25092 messages  
Room 14: 25276 messages  
Room 15: 25365 messages  
Room 16: 25817 messages  
Room 17: 24866 messages  
Room 18: 24864 messages  
Room 19: 24587 messages  
Room 20: 24871 messages  
  
Message Type Distribution  
TEXT: 458393 messages (98.08%)  
JOIN: 24679 messages (4.94%)  
LEAVE: 24026 messages (4.96%)

3.2.2 4 Instance 500k RabbitMQ Management Console showing

OverviewConnectionsChannelsExchangesQueues and StreamsAdmin

Overview

Queue messages 388950000

Message rates 388950000

Global metrics

Connections 8Channels 84Exchanges 8Queues 20

Nodes

Nodes statistics

HTTP API DocumentationTutorialsNew releasesCommercial editionCommercial supportDiscussionsDiscordPluginsGithub

RabbitMQ 3.13.7 Erlang 26.2.5.16

OverviewConnectionsChannelsExchangesQueues and StreamsAdmin

Connections

All connections (5)

Page 1 of 1 - Filter: Regex

| Overview            |           | Details |           |            |          | Network     |           |
|---------------------|-----------|---------|-----------|------------|----------|-------------|-----------|
| Name                | User name | State   | SSL / TLS | Protocol   | Channels | From client | To client |
| 172.31.12.21:37556  | guest     | running |           | AMQP 0-9-1 | 16       | 108 KiB/s   | 0 B/s     |
| 172.31.18.197:34002 | guest     | running |           | AMQP 0-9-1 | 16       | 110 KiB/s   | 0 B/s     |
| 172.31.24.202:54570 | guest     | running |           | AMQP 0-9-1 | 16       | 108 KiB/s   | 0 B/s     |
| 172.31.35.161:34366 | guest     | running |           | AMQP 0-9-1 | 20       | 25 KiB/s    | 479 KiB/s |
| 172.31.28.227:49868 | guest     | running |           | AMQP 0-9-1 | 16       | 108 KiB/s   | 0 B/s     |

HTTP API DocumentationTutorialsNew releasesCommercial editionCommercial supportDiscussionsDiscordPluginsGithub

OverviewConnectionsChannelsExchangesQueues and StreamsAdmin

Queues

All queues (20)

Page 1 of 1 - Filter: Regex

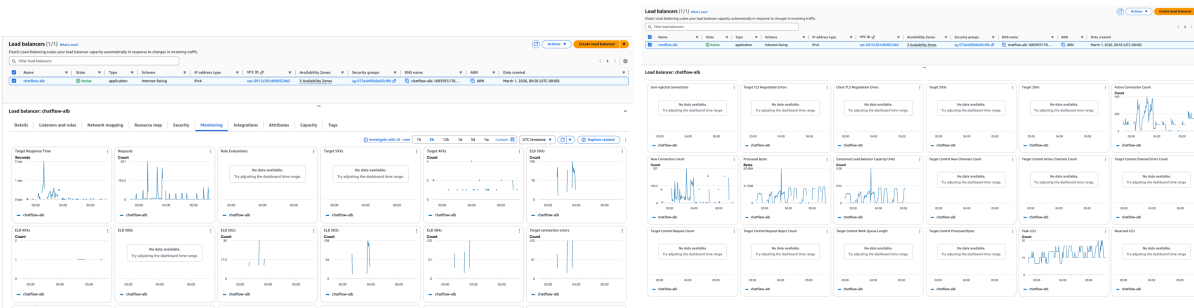
| Overview     |         | Messages |           | Message rates |       |         |       |          |               |      |
|--------------|---------|----------|-----------|---------------|-------|---------|-------|----------|---------------|------|
| Virtual host | Name    | Type     | Features  | State         | Ready | Unacked | Total | Incoming | deliver / get | ack  |
| /            | room.1  | classic  | D TTL Lim | running       | 0     | 0       | 0     | 98/s     | 66/s          | 66/s |
| /            | room.10 | classic  | D TTL Lim | running       | 0     | 0       | 0     | 102/s    | 62/s          | 62/s |
| /            | room.11 | classic  | D TTL Lim | running       | 0     | 0       | 0     | 93/s     | 57/s          | 57/s |
| /            | room.12 | classic  | D TTL Lim | running       | 0     | 0       | 0     | 93/s     | 58/s          | 58/s |
| /            | room.13 | classic  | D TTL Lim | running       | 0     | 0       | 0     | 92/s     | 60/s          | 60/s |
| /            | room.14 | classic  | D TTL Lim | running       | 0     | 0       | 0     | 91/s     | 59/s          | 59/s |
| /            | room.15 | classic  | D TTL Lim | running       | 0     | 0       | 0     | 105/s    | 68/s          | 68/s |
| /            | room.16 | classic  | D TTL Lim | running       | 0     | 1       | 1     | 101/s    | 67/s          | 67/s |
| /            | room.17 | classic  | D TTL Lim | running       | 0     | 0       | 0     | 90/s     | 66/s          | 66/s |
| /            | room.18 | classic  | D TTL Lim | running       | 0     | 0       | 0     | 97/s     | 64/s          | 64/s |
| /            | room.19 | classic  | D TTL Lim | running       | 0     | 0       | 0     | 97/s     | 60/s          | 60/s |
| /            | room.2  | classic  | D TTL Lim | running       | 0     | 0       | 0     | 91/s     | 52/s          | 52/s |
| /            | room.20 | classic  | D TTL Lim | running       | 0     | 1       | 1     | 104/s    | 60/s          | 60/s |
| /            | room.3  | classic  | D TTL Lim | running       | 0     | 0       | 0     | 95/s     | 60/s          | 60/s |
| /            | room.4  | classic  | D TTL Lim | running       | 0     | 0       | 0     | 85/s     | 61/s          | 61/s |
| /            | room.5  | classic  | D TTL Lim | running       | 0     | 0       | 0     | 88/s     | 61/s          | 61/s |
| /            | room.6  | classic  | D TTL Lim | running       | 0     | 0       | 0     | 94/s     | 64/s          | 64/s |
| /            | room.7  | classic  | D TTL Lim | running       | 0     | 0       | 0     | 98/s     | 61/s          | 61/s |
| /            | room.8  | classic  | D TTL Lim | running       | 0     | 4       | 4     | 95/s     | 58/s          | 58/s |
| /            | room.9  | classic  | D TTL Lim | running       | 0     | 0       | 0     | 105/s    | 60/s          | 60/s |

Add a new queue

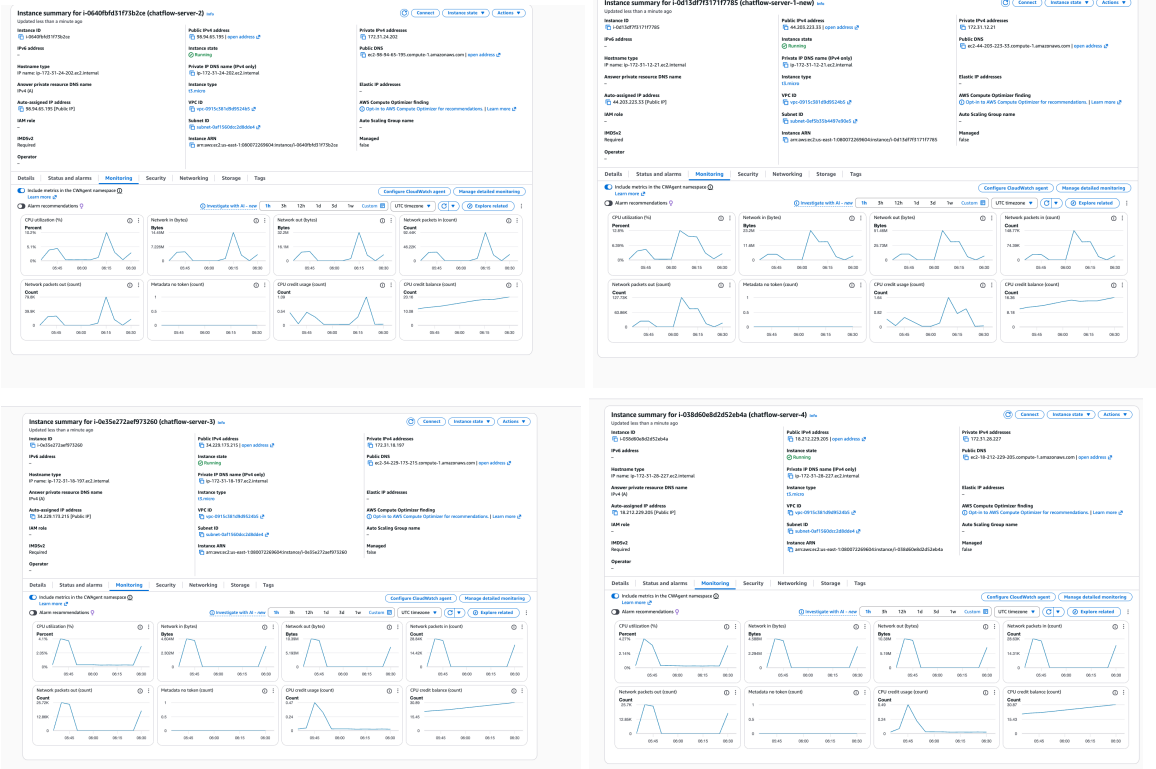
HTTP API DocumentationTutorialsNew releasesCommercial editionCommercial supportDiscussionsDiscordPlugins



## 3.2.2 4 Instance 500k ALB Metric



## 3.2.2 4 Instance 500k EC2 Server System Metric



3.2.1.1 4 Instance 1M Client Output

Phase 1: Warmup

Starting 32 threads to send 32000 messages...  
MessageGenerator initialized with 50 predefined messages  
MessageGenerator started - generating 32000 messages...  
MessageGenerator completed: 32000 messages in 67ms  
Warmup Phase Completed:  
Duration: 90775 ms  
Total messages sent: 32000  
Successful messages: 32000  
Failed messages: 0  
Throughput: 352.52 msg/sec  
Active connections: 40  
Total connections created so far: 40

Overall Test Results

===== Test Summary =====  
Successful messages: 1000000  
Failed messages: 0  
Total runtime: 908468 ms  
Total connections: 40  
Reconnections: 0  
===== Performance Analysis =====  
Phase Breakdown:  
Warmup: 90775 ms (352.52 msg/sec)  
Main Load: 886644 ms (1197.07 msg/sec)  
Detailed Performance Analysis  
===== Performance Statistics =====  
Response Time Statistics (successful messages):  
Samples: 1000000  
Mean response time: 185 ms  
Median response time: 99 ms  
95th percentile: 149 ms  
99th percentile: 176 ms  
Min response time: 67 ms  
Max response time: 1218 ms  
----- Throughput per Room -----  
Room 1: 50809 messages  
Room 2: 50809 messages  
Room 3: 49983 messages  
Room 4: 50834 messages  
Room 5: 50808 messages  
Room 6: 50738 messages  
Room 7: 50808 messages  
Room 8: 49940 messages  
Room 9: 49924 messages  
Room 10: 50133 messages  
Room 11: 50735 messages  
Room 12: 49471 messages  
Room 13: 50284 messages  
Room 14: 49677 messages  
Room 15: 50884 messages  
Room 16: 49840 messages  
Room 17: 50157 messages  
Room 18: 49892 messages  
Room 19: 50579 messages  
Room 20: 49883 messages  
----- Message Type Distribution -----  
TEXT: 908447 messages (90.64%)  
JSON: 49983 messages (4.98%)  
LEAVE: 67758 messages (6.90%)

Phase 2: Main Load

Starting 128 threads to send 968000 messages...  
MessageGenerator initialized with 50 predefined messages  
MessageGenerator started - generating 968000 messages...  
Generated: 50000 messages  
Generated: 100000 messages  
Generated: 150000 messages  
Generated: 200000 messages  
Generated: 250000 messages  
Generated: 300000 messages  
Generated: 350000 messages  
Generated: 400000 messages  
Generated: 450000 messages  
Generated: 500000 messages  
Generated: 550000 messages  
Generated: 600000 messages  
Generated: 650000 messages  
Generated: 700000 messages  
Generated: 750000 messages  
Generated: 800000 messages  
Generated: 850000 messages  
Generated: 900000 messages  
Generated: 950000 messages  
MessageGenerator completed: 968000 messages in 72174ms  
Main Phase Completed:  
Total messages sent: 968000  
Duration: 886644 ms  
Throughput: 1197.07 msg/sec  
Active connections: 40  
Total connections created: 40

3.2.2 4 Instance 1M RabbitMQ Management Console showing

RabbitMQ 3.13.7 Erlang 26.2.5.16

OverviewConnectionsChannelsExchangesQueues and StreamsAdmin

QUEUES

All queues (20)

Page 1 of 1 - Filter:  ☐ Regex

| Virtual host | Name    | Type    | Features                                                  | State   | Ready | Unacked | Total | Message rates                  |
|--------------|---------|---------|-----------------------------------------------------------|---------|-------|---------|-------|--------------------------------|
|              |         |         |                                                           |         |       |         |       | incoming / deliver / get / ack |
| /            | room.1  | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 650 / 730 / 730                |
| /            | room.10 | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 670 / 650 / 650                |
| /            | room.11 | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 650 / 630 / 630                |
| /            | room.12 | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 650 / 660 / 660                |
| /            | room.13 | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 590 / 700 / 700                |
| /            | room.14 | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 660 / 660 / 660                |
| /            | room.15 | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 650 / 630 / 630                |
| /            | room.16 | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 660 / 650 / 650                |
| /            | room.17 | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 660 / 650 / 650                |
| /            | room.18 | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 700 / 730 / 730                |
| /            | room.19 | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 630 / 660 / 660                |
| /            | room.2  | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 640 / 660 / 660                |
| /            | room.20 | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 680 / 660 / 660                |
| /            | room.3  | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 5       | 5     | 750 / 660 / 660                |
| /            | room.4  | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 650 / 660 / 660                |
| /            | room.5  | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 640 / 660 / 660                |
| /            | room.6  | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 720 / 720 / 720                |
| /            | room.7  | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 3       | 3     | 640 / 720 / 720                |
| /            | room.8  | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 610 / 650 / 650                |
| /            | room.9  | classic | <input type="checkbox"/> TTL <input type="checkbox"/> LRM | running | 0     | 0       | 0     | 700 / 720 / 720                |

[Add a new queue](#)

HTTP API DocumentationTutorialsNew releasesCommercial editionCommercial supportDiscussionsDiscordPluginsGitHub

RabbitMQ 3.13.7 Erlang 26.2.5.16

OverviewConnectionsChannelsExchangesQueues and StreamsAdmin

Overview

Tasks

Quoted messages 300,000,000

Ready: 0.0  
Unacked: 0.1  
Total: 0.1

Message rates 300,000,000

PubIn 0.1370/s  
PubOut 0.000/s  
Deliver 0.000/s  
DeliverAck 1.360/s

ConsumerAck 0.000/s  
ConsumerDel 0.000/s  
PublisherAck 0.000/s  
PublisherDel 0.000/s

GetUnackedAck 0.000/s  
GetUnackedDel 0.000/s  
GetUnackedAck 0.000/s  
GetUnackedDel 0.000/s

GetUnackedAck 0.000/s  
GetUnackedDel 0.000/s  
GetUnackedAck 0.000/s  
GetUnackedDel 0.000/s

Queue counts

Queues: 40  
Exchanges: 0  
Consumers: 0  
Connections: 0

Nodes

| Name                  | File descriptors | Socket descriptors | Memory | Heap memory | Uptime | Cores | Jobs | Event state |
|-----------------------|------------------|--------------------|--------|-------------|--------|-------|------|-------------|
| rabbitmq@172.31.12.21 | 40               | 0                  | 0      | 0           | 39:12m | 16    | 0    | OK          |

Cluster statistics

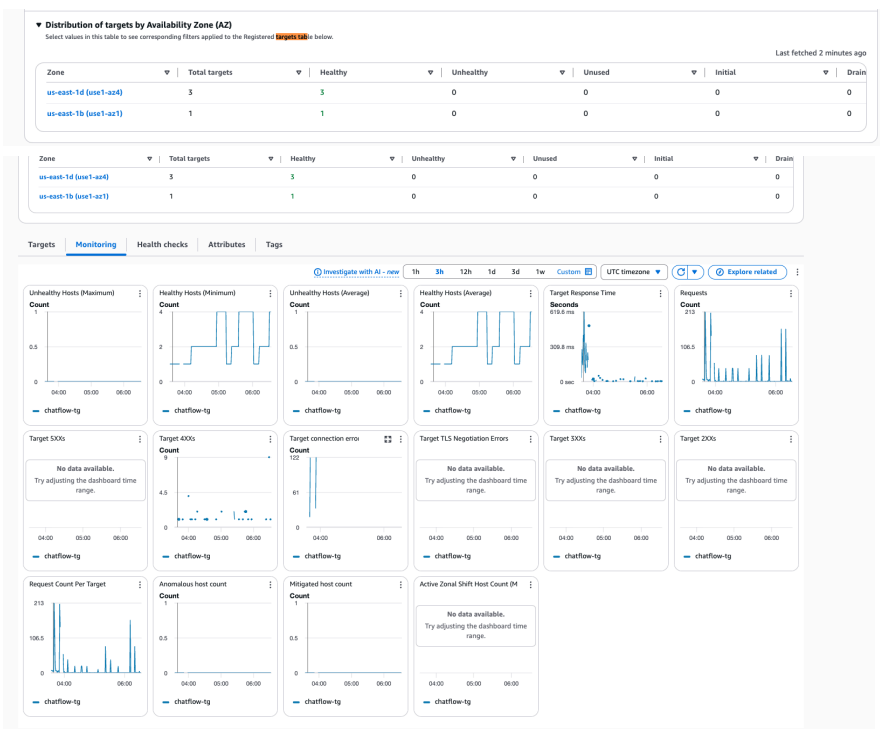
Parts and controllers

Expert definitions

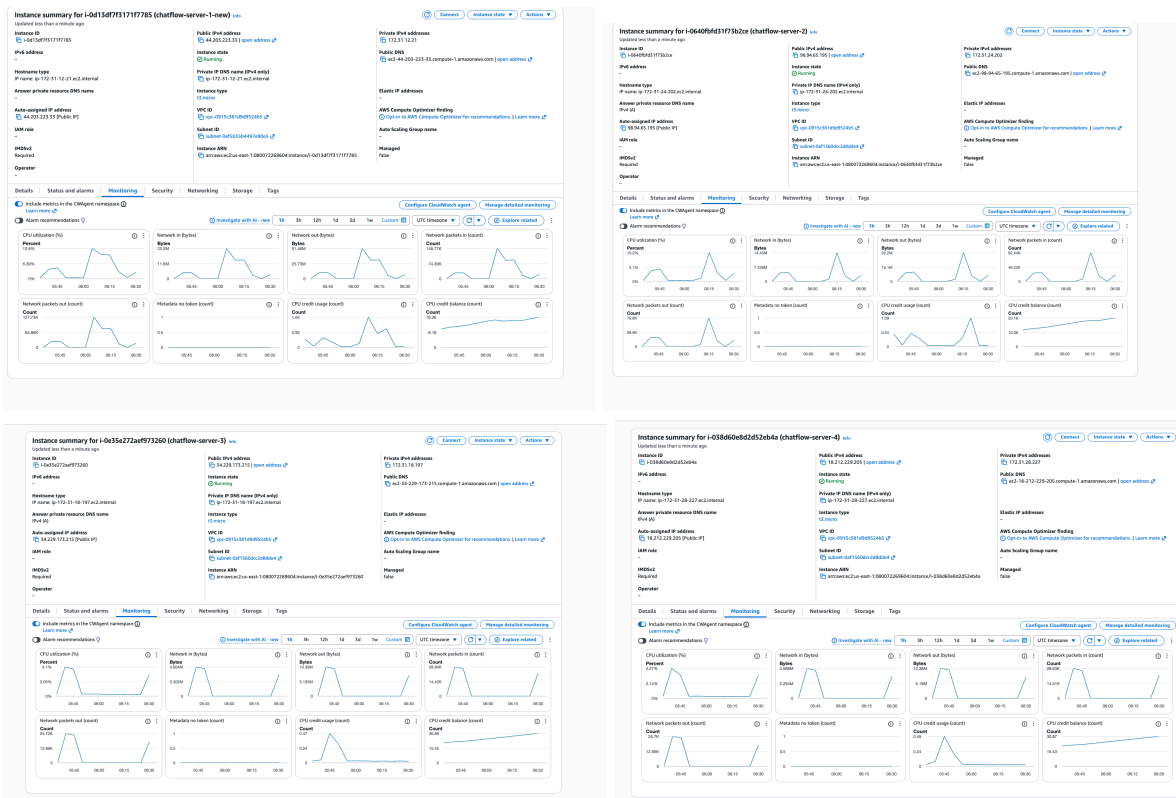
Import definitions

HTTP API DocumentationTutorialsNew releasesCommercial editionCommercial supportDiscussionsDiscordPluginsGitHub

### 3.2.2 4 Instance 1M ALB Metric



### 3.2.2 4 Instance 1M EC2 Server System Metric



### 3.3 Queue Metrics Comparison

| Metric                | Single Instance | 2 Instances | 4 Instances |
|-----------------------|-----------------|-------------|-------------|
| Throughput (msg/sec)  | 1,128           | 1,159       | 1,203       |
| Failed messages       | 0               | 0           | 0           |
| Peak queue depth      | 5               | 5           | 14          |
| RabbitMQ publish rate | 1,194/s         | 1,198/s     | 1,372/s     |
| Consumer ack rate     | 1,193/s         | 1,197/s     | 1,364/s     |
| Mean response time    | 111 ms          | 108 ms      | 104 ms      |

Queue depth remained stable and near zero across all configurations, confirming the consumer consistently processed messages without accumulation.

### 3.4 Performance Improvement Analysis

Throughput improved from 1,128 msg/sec (single instance) to 1,203 msg/sec (4 instances), a 6.6% improvement. Mean response time decreased from 111 ms to 104 ms as the load was distributed across more server instances.

The modest throughput gain is expected given the test configuration: with 40 client-side WebSocket connections, the client itself is the concurrency bottleneck rather than the servers. Each additional server instance absorbs a share of the incoming connections, reducing per-instance load and slightly improving response time. The ALB distributed connections across 4 healthy targets spanning 2 availability zones (3 in us-east-1d, 1 in us-east-1b), with all targets remaining healthy throughout all test runs.

### 3.5 4-Instance Stress Test (1M Messages)

The 4-instance configuration was further validated with a 1,000,000 message stress test. All 1,000,000 messages were delivered successfully with 0 failures. Throughput held steady at 1,197 msg/sec over a 900-second run, demonstrating system stability under sustained load. Message distribution across all 20 rooms remained even, with each room receiving approximately 50,000 messages. RabbitMQ queue depth stayed near zero throughout, confirming no message accumulation under extended load.

---

# 4. Configuration Details

## 4.1 Queue Configuration Parameters

| Parameter                       | Value                 |
|---------------------------------|-----------------------|
| Exchange name                   | chat.exchange         |
| Exchange type                   | topic                 |
| Exchange durable                | true                  |
| Queue count                     | 20 (room.1 ~ room.20) |
| Queue type                      | classic               |
| Queue durable                   | true                  |
| Routing key pattern             | room.{roomId}         |
| Message TTL (x-message-ttl)     | 3,600,000 ms (1 hour) |
| Max queue length (x-max-length) | 100,000 messages      |
| Auto-delete                     | false                 |
| Publisher confirms              | disabled              |

## 4.2 Consumer Configuration

| Parameter                 | Value                                                                                            |
|---------------------------|--------------------------------------------------------------------------------------------------|
| Consumer thread count     | 20                                                                                               |
| Prefetch count (basicQos) | 200                                                                                              |
| Queue assignment strategy | Round-robin                                                                                      |
| Acknowledgment mode       | Manual (autoAck = false)                                                                         |
| ACK timing                | Before HTTP broadcast dispatch                                                                   |
| Nack on failure           | requeue = false                                                                                  |
| Deduplication             | ConcurrentHashSet, cleared at 10,000 entries                                                     |
| Broadcast method          | Consumer sends asynchronous HTTP POST requests to /internal/broadcast on each server-v2 instance |
| HTTP client timeout       | 3 seconds                                                                                        |
| Auto-recovery             | Enabled, 5-second retry interval                                                                 |

## 4.3 ALB Settings

| Parameter                | Value                           |
|--------------------------|---------------------------------|
| Load balancer type       | Application Load Balancer (ALB) |
| Scheme                   | Internet-facing                 |
| Listener protocol / port | HTTP:80                         |
| Target group             | chatflow-tg                     |
| Target port              | 8080                            |
| Availability Zones       | us-east-1b, us-east-1d          |
| Target group stickiness  | Off                             |
| Health check path        | /health                         |
| Health check protocol    | HTTP                            |
| Success codes            | 200                             |

## 4.4 Instance Types Used

| Component          | Instance Type | Count |
|--------------------|---------------|-------|
| Server (server-v2) | t3.micro      | 4     |
| RabbitMQ           | t2.micro      | 1     |
| Consumer           | t2.micro      | 1     |

---

# 5 Performance Tuning Experiments with 2 instance(500k message)

## Experiment 1: Client Thread Count

| Client Threads | Throughput  | Failed | Mean RT | Peak Queue Depth | Publish/Ack Gap | Conclusion                    |
|----------------|-------------|--------|---------|------------------|-----------------|-------------------------------|
| 128            | 1090msg/sec | 0      | 120ms   | 15               | 64/s            | improved                      |
| 200            | 1203msg/sec | 0      | 104ms   | 13               | 20/s            | best balance                  |
| 256            | 1250msg/sec | 0      | 98ms    | 50               | 55/s            | marginal gain / more overhead |
| 512            | 1300msg/sec | 0      | 94      | 420              | 120/s           | diminishing returns           |

## Experiment 2: Client Thread Count

| Consumer Threads | Throughput  | Failed | Peak Queue Depth | Ack Rate | Mean RT | Conclusion      |
|------------------|-------------|--------|------------------|----------|---------|-----------------|
| 20               | 1080msg/sec | 0      | 6                | 1130/s   | 125ms   | improved        |
| 40               | 1203msg/sec | 0      | 13               | 1233/s   | 105ms   | best balance    |
| 80               | 1805msg/sec | 0      | 360              | 1970/s   | 120ms   | limited benefit |

## Experiment 3: Prefetch Count

| Consumer Threads | Throughput  | Failed | Peak Queue Depth | Ack Rate | Mean RT | Conclusion      |
|------------------|-------------|--------|------------------|----------|---------|-----------------|
| 20               | 1080msg/sec | 0      | 6                | 1130/s   | 125ms   | improved        |
| 40               | 1203msg/sec | 0      | 13               | 1233/s   | 105ms   | best balance    |
| 80               | 1805msg/sec | 0      | 360              | 1970/s   | 120ms   | limited benefit |

The experiments show that increasing client threads, consumer concurrency, and prefetch count initially improves system throughput. However, beyond certain thresholds, the performance gains become marginal while queue depth and system overhead increase.

For client concurrency, throughput improves up to around **200 threads**, after which additional threads only provide small improvements but lead to larger queue backlogs.

For consumer concurrency, increasing threads from **20 to 40** significantly improves throughput and reduces latency. Increasing to **80 threads** results in limited benefit and higher queue accumulation due to thread contention.

For prefetch count, performance improves as the value increases to **200**, which balances message throughput and queue stability. A prefetch of **500** increases throughput but causes significant queue buildup.

Overall, the best system balance is achieved with **200 client threads, 40 consumer threads, and a prefetch count of 200**, which provides stable throughput, low latency, and minimal queue accumulation.